Field Hours

CSCE 752 Robotics and Spatial Intelligence

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Frames and rigid transforms

A frame is an origin and three orthogonal axes. You write a point as (1, 0, 0) in the sensor frame, and it is not that triple in the world until you apply the rigid transform. The transform is a rotation plus a translation. If the sensor origin sits at world (2, 0, 0) and the axes match, that point becomes (3, 0, 0). You apply the nearer frame first when you compose two transforms.

You store the transform as a 4 by 4 matrix on homogeneous coordinates. A point ends in 1 and a direction ends in 0, so translation is multiplication and the last row stays 0 0 0 1. You invert the matrix to take a parent-frame point back into the child. A translation error of 0.02 meters biases every mapped point by that same 0.02 meters. You cannot average that bias away, because it is not noise.

Express one point in the world.

Read sensor coordinates -> Apply the rotation -> Add the origin translation -> Attach the world frame

Aligned axes, a shifted origin

1. The sensor origin is at world (2, 0, 0), and the axes match the world axes. The point in the sensor frame is (1, 0, 0).

2. World coordinates are (2, 0, 0) plus (1, 0, 0), so (3, 0, 0).

3. Adding the translation twice reports (5, 0, 0), which is off by 2 meters. Composition is not a second copy of the same vector.

Problem 1. What are the world coordinates of the sensor point (1, 0, 0) in the example?

Answer. (3, 0, 0).

Problem 2. Every mapped point is biased by 0.02 meters along x, and the noise on each scan is about 0.001 meters. Why does averaging 100 scans not remove the bias?

Answer. The 0.02 meter shift is in the rigid transform, so it is common to every scan. Averaging shrinks the 0.001 meter noise. It does not remove an error that is the same each time.

Lecture 2 · 90 minutes

Forward and inverse kinematics

Forward kinematics multiplies one rigid transform per joint, from the base out to the tip. A planar arm with two links of length 1, at 0 degrees and 90 degrees, puts the tip at (1, 1) if 0 degrees points along x. A 6-joint arm is the same product with 6 factors. You evaluate that map with no solver. It is a function of joint numbers you

already have.

Inverse kinematics asks for joints that hit a desired pose. For this 2-link arm the target (1, 1) has 2 solutions, because the elbow can bend either way. A target at distance 3 is unreachable: two unit links stretch at most 2. You test that bound before you iterate. A numerical solver can settle on the nearer legal solution, and you cap it, for example at 50 steps, when the point is unreachable.

A planar two-link chain.

Base frame

Joint 1 then a unit link

Joint 2 then a unit link

End-effector pose

Two solutions, and one empty set

1. Links have length 1. Joints at 0 degrees and 90 degrees put the tip at (1, 1). The distance from the base is the square root of 2, about 1.414, which is under 2, so the point is reachable.

2. The other solution uses 90 degrees and then -90 degrees. The first link ends at (0, 1). The second link then points along positive x and ends at (1, 1).

3. The target (3, 0) is 3 from the base. No pair of unit-length links reaches it. Iteration does not create a configuration the geometry does not have.

Problem 1. Where is the tip for joints 0 degrees and 90 degrees, and what other joint pair in the lecture hits the same point?

Answer. The tip is at (1, 1). The other pair is 90 degrees and -90 degrees.

Problem 2. Why does a solver have no answer for the target (3, 0)?

Answer. Two links of length 1 reach at most 2. The target is 3 from the base, outside the workspace. No inverse solution exists.

Lecture 3 · 90 minutes

Jacobians and tip velocity

The Jacobian is the derivative of forward kinematics. For this planar arm it is 2 by 2 if you track the tip in x and y. You multiply it by joint rates in radians per second and get tip velocity in meters per second. At 0 degrees and 90 degrees, with unit links, the rows are (-1, -1) and (1, 0). A zero column would mean that joint does not move the tip at first order.

A singular pose drops the rank, so some tip directions are instantaneously impossible. Both angles at 0 stretch the arm to (2, 0). The columns become parallel and the determinant is 0. You cannot produce a first-order velocity that lengthens the arm further. A damped solver adds about 0.01 on the diagonal before inverting, trading a few millimeters of error for a finite command. You do not invert a matrix whose determinant is 0.

A usable pose and a singular one.

0 and 90 degrees

Determinant equals 1

Second column is (-1, 0)

Joint rate maps to tip speed

Both angles 0

Determinant equals 0

Columns both point along y

Radial velocity has no solution

One joint rate

1. Use the Jacobian with rows (-1, -1) and (1, 0), at 0 degrees and 90 degrees.

2. A joint rate of (0, 1) radian per second selects the second column.

3. The tip velocity is (-1, 0) meters per second. The determinant of this matrix is 1, so the pose is not singular. The stretched pose, both angles 0, has determinant 0 and is a different question.

Problem 1. At 0 degrees and 90 degrees, what tip velocity does the joint rate (0, 1) radian per second produce?

Answer. (-1, 0) meters per second, the second column of the Jacobian.

Problem 2. Both joints are at 0 degrees. Why is a requested tip velocity of (1, 0) not achievable at first order?

Answer. The arm is fully stretched on the x axis and the determinant is 0. Both columns point along y, so first-order tip motion is vertical. A command along positive x has no solution. Damping can return a finite joint rate, and it will not produce that radial velocity exactly.

Lecture 4 · 90 minutes

Planning in free space

You search pose coordinates for a collision-free path. On a grid you mark occupied cells and run a shortest-path search. A 4-connected grid charges 1 per step and forbids diagonals, so 3 cells east and 4 north cost 7 when free. An 8-connected grid prices diagonals differently, and you must say which graph you built. The path stays on free cells. It does not cut a corner through an occupied cell.

A random tree grows toward samples. You draw a configuration, find the nearest node, and step at most 0.5 meters, rejecting the step if it hits an obstacle. With 200 accepted nodes you often reach a goal a coarse grid missed, and the path is often longer than the grid optimum because the tree does not price cost the same way. The first feasible path is not a shortest path you should quote as optimal.

One tree expansion.

1. Sample a configuration

2. Find the nearest node

3. Step at most 0.5 meters

4. Add the node only if free

A blocked cell forces a detour

1. The grid is 4-connected. Start at (0, 0). The goal is (2, 0). The cell (1, 0) is blocked, so the straight graph path of length 2 is illegal.

2. One free path is (0, 0), (0, 1), (1, 1), (2, 1), (2, 0). Its length is 4.

3. The extra cost relative to the blocked straight path is 2. A tree that later returns a feasible path of length 6 has not contradicted this grid result. It answered a different question.

Problem 1. What is the length of the detour through (0, 1), (1, 1), and (2, 1), and by how much does it exceed the blocked straight graph distance?

Answer. The detour has length 4. It exceeds 2 by 2.

Problem 2. A random tree returns a feasible path of length 6 in this same world. Why is that not a counterexample to the grid path of length 4?

Answer. The tree returns a feasible path, not a shortest path. The 4-connected search optimized length and found 4. A longer feasible path can be a correct output of the tree.

Lecture 5 · 90 minutes

Calibration

Camera calibration recovers the map from a pixel to a ray. In a pinhole model, a point 2 meters in front of the camera that lands 200 pixels right of the principal point, with focal length 400 pixels, has lateral slope 200 divided by 400, so 0.5. You estimate focal length, principal point, and a few distortion coefficients from a known pattern, often 10 or more views of a checkerboard. One view leaves those parameters tangled with the unknown pose.

Hand-eye calibration finds the rigid transform between the flange and a camera fixed on it. You record 8 flange poses and the camera view of a fixed board. The two motions are one physical motion written in two frames. A fit from only 2 poses is usually noisy. You report millimeters of residual on a pose you held out. Error of 1 millimeter on the 8 fits and 15 on a ninth pose means the solver memorized the set.

Two calibrations you do not mix up.

Camera intrinsics

Focal length in pixels

Principal point

A few distortion coefficients

Hand-eye transform

Flange poses from the arm

Camera poses from the board

Residual on a held-out pose

Pixels to a metric offset

1. Focal length is 400 pixels. The detection sits 200 pixels from the principal point. The ray slope is 200 divided by 400, so 0.5.

2. At a depth of 2 meters the lateral position is 0.5 times 2, so 1 meter.

3. A focal length of 800 pixels would turn the same 200 pixel offset into a slope of 0.25 and a lateral position of 0.5 meters at that depth. A factor-of-2 error in focal length doubles the metric position you report.

Problem 1. Focal length 400, pixel offset 200, depth 2 meters. What lateral position do you reconstruct?

Answer. 1 meter. The slope is 0.5, and 0.5 times 2 is 1.

Problem 2. Hand-eye residual is 1 millimeter on the 8 fitting poses and 15 millimeters on a held-out pose. Which number do you trust as the calibration error?

Answer. The 15 millimeter held-out residual. The fitting poses are the data the solver matched. A small residual there does not measure a new pose.

Lecture 6 · 90 minutes

Stereo depth

A calibrated stereo pair recovers depth from disparity. With a baseline of 0.2 meters, a focal length of 400 pixels, and a disparity of 8 pixels, the depth is baseline times focal length divided by disparity, so 0.2 times 400 divided by 8, which is 10 meters. Doubling the disparity to 16 pixels halves the depth to 5 meters. After rectification you match along the epipolar line, a one-dimensional search rather than a search over the whole image.

A fixed pixel mistake hurts more at long range. At disparity 8, a 1 pixel error uses disparity 9 and moves depth from 10 meters to 0.2 times 400 divided by 9, about 8.89 meters, a shift of about 1.11 meters. At disparity 40 the depth is 2 meters, and disparity 41 gives about 1.95 meters, a shift of about 0.05 meters. You do not quote one millimeter accuracy for the whole volume. The distant match is the weak one.

From a matched pair to a depth.

Rectify the image pair -> Match along one row -> Read the disparity in pixels -> Divide baseline times focal length

One pixel, two ranges

1. Baseline 0.2 meters and focal length 400 pixels make the product 80. Disparity 8 gives depth 80 divided by 8, so 10 meters.

2. Disparity 9 gives 80 divided by 9, about 8.89 meters. The absolute error is about 1.11 meters.

3. Disparity 40 gives 2 meters. Disparity 41 gives 80 divided by 41, about 1.95 meters. The absolute error is about 0.05 meters. The same 1 pixel mistake is not the same metric mistake.

Problem 1. Baseline 0.2 meters, focal length 400 pixels, disparity 8. What is the depth?

Answer. 10 meters.

Problem 2. A matcher is off by 1 pixel. At which disparity, 8 or 40, is the depth error larger, and about how many meters is each error?

Answer. Disparity 8 is worse, about 1.11 meters (10 versus about 8.89). Disparity 40 errs by about 0.05 meters (2 versus about 1.95).

Lecture 7 · 90 minutes

SLAM and navigation

You estimate pose and map together, because each depends on the other. A pose graph stores poses as nodes and motions or loop closures as edges. With 100 poses and 1 loop closure, that edge makes the drift observable. Odometry high by 0.01 meters on every step accumulates about 1 meter after 100 steps when the errors share a sign. The optimizer moves the nodes to cut the sum of squared edge errors.

You then localize, plan on the free grid, and track the path. A local planner that looks only 2 meters ahead can dodge a new chair, and it can enter a dead end a global plan of 40 meters already rejected. You keep both. A pose whose uncertainty spans 3 meters is not exact enough when the doorway is only 1 meter wide.

From a loop edge to a path.

1. Add the odometry edges

2. Add the loop-closure edge

3. Optimize the pose graph

4. Plan on the resulting map

One meter of shared bias

1. Each true step is 1.00 meter. Each odometry edge reads 1.01 meters. After 100 steps the chain sums to 101 meters, while the truth is 100.

2. A loop closure back to the start measures a relative pose near 0. The graph now contains a 1 meter disagreement.

3. If the optimizer spreads that 1 meter evenly, each edge gives up 0.01 meter and the chain matches the closure. You still should not drive a mean pose through a 1 meter doorway when the pose uncertainty spans 3 meters.

Problem 1. After 100 steps biased by 0.01 meter each, how far does the odometry chain say you traveled if each true step is 1 meter, and how large is the disagreement with a zero loop closure?

Answer. The chain sums to 101 meters. The disagreement with a return to the start is 1 meter.

Problem 2. Why can a planner that only sees 2 meters ahead enter a dead end that a 40 meter global

plan avoided?

Answer. The local window does not contain the trap if the failure sits beyond 2 meters. The global plan scored the whole 40 meter route and rejected it. Using only the local planner drops that information.

Lecture 8 · 90 minutes

A stack you can audit

You can now move a point between frames, multiply a two-link arm, and reject a target at distance 3 when the links reach only 2. You can read a tip velocity off a Jacobian, search a 4-connected grid, and grow a tree with a 0.5 meter step. You can turn a pixel offset and a focal length into a metric ray, turn a disparity into a depth, and pull a 1 meter odometry disagreement back with one loop closure.

You can name the number that makes the stack fail. A transform bias of 0.02 meters is not scan noise. Stereo at disparity 8 moves about 1.11 meters if the match slips by 1 pixel, while disparity 40 moves about 0.05 meters. A pose uncertainty of 3 meters does not fit a doorway of 1 meter. The system you sketch is a calibrated sensor, a map, a plan, and a check that the uncertainty still fits the gap.

The navigation stack, with the check.

Calibrate the sensor frame -> Build a map and close a loop -> Plan on an inflated grid -> Reject gaps inside the uncertainty

Depth error versus planner inflation

1. Baseline 0.2 meters, focal length 400 pixels, and disparity 8 put an obstacle at 10 meters. A 1 pixel slip moves that depth by about 1.11 meters, from the stereo lecture.

2. The planner inflates obstacles by 0.3 meters. That radius is smaller than 1.11. A path that clears the nominal point by 0.3 meters can still hit the obstacle.

3. You inflate by at least that depth error, or you refuse the far match. The grid cost is then computed on the inflated map. A transform bias of 0.02 meters is a separate error, and the pose-graph update does not cancel it.

Problem 1. Combine stereo depth with planner inflation. Disparity 8 gives depth 10 meters, a 1 pixel mistake moves depth by about 1.11 meters, and inflation is 0.3 meters. Why is the clearance insufficient?

Answer. Inflation covers 0.3 meters, and the disparity error moves the obstacle by about 1.11 meters. A path that clears the nominal point by 0.3 meters can intersect the true obstacle. The inflation needs to be at least on the order of that depth error, or the far match should be refused.

Problem 2. Combine frames and the loop closure. The sensor-to-base translation is wrong by 0.02 meters, and the pose graph sees a 1 meter odometry disagreement after 100 steps. Which error can the optimizer remove, and which remains?

Answer. The optimizer can spread the 1 meter odometry disagreement across the edges. The 0.02 meter sensor-to-base bias sits in the rigid transform outside those edges, so mapped points stay shifted by 0.02 meters.